

An Investigation on Word Embedding Offset Clustering as Relationship Classification

Didier Gohourou and Kazuhiro Kuwabara

Ritsumeikan University

Abstract. Vector representations obtained from word embedding are the source of many groundbreaking advances in natural language processing. They yield word representations that are capable of capturing semantics and analogies of words within a text corpus. This study is an investigation in an attempt to elicit a vector representation of relationships between pairs of word vectors. We use six pooling strategies to represent vector relationships. Different types of clustering models are applied to analyze which one correctly groups relationship types. Subtraction pooling coupled with a centroid based clustering mechanism shows better performances in our experimental setup. This work aims to provide directions for a word embedding based unsupervised method to identify the nature of a relationship represented by a pair of words.

Keywords: Word Embedding · Clustering · Relationship Representation

1 Introduction

The study of pattern recognition in natural language processing requires a numerical representation of text. This is achieved by computing a vector representation for the words that compose text corpora. Words were originally represented using a one-hot encoding vector representation. The approach had shortcomings including sparsed and high dimensional vectors that are unable to capture the contextual meaning of words. Proposed by Bengio et al, word embedding is a neural language model that addresses the *curse of dimensionality*. The proposed model also provides a distributed representation of words, aligned with linguistic principles such as the context-dependent nature of meaning. Different implementations have spawned from the proposition including word2vec [7,4], GloVe [10], and fastText [1]. Those word representation models are the backbone of many natural language processing (NLP) tasks including named entity recognition, sentiment analysis, and machine translation, which led to state-of-the-art breakthroughs in those respective NLP fields.

Word embedding demonstrated additional properties, such as the ability to capture syntactic and semantic regularities in language [8]. Building from this insight, can we obtain vectors that can capture the type of relationship between word embedding? Can those relationship representations be effectively grouped with clustering models? This study is an attempt to answer those questions. It

explores different *pooling* approaches to obtain a vector that represents the relationship between word vectors based on their embedding. The study also uses different clustering models in an attempt to score the ability of the relationship vectors to be grouped. Answering those problems will point toward *an unsupervised methodology to classify relationships between words*. Our contributions can be emphasized as follows:

- Explore for a word embedding representation of the relationship for a pair of words.
- Analyze the clustering of those relationship vectors.
- Cue toward an unsupervised classification mechanism of relationships between words.

The rest of the study unfolds by first discussing related works including the clustering of word vectors and the elicitation of regularities in word vector space. Then present the methodologies adopted for word embedding relationship representation and the descriptions of the families of clustering algorithms selected. An experiment section follows to detail the data set and specific cluster algorithms used. A discussion analyzes the results and offers directions to apply the findings, before concluding the study.

2 Related Work

Vector-space word representations learned by continuous space language models such as word2vec models have demonstrated the ability to capture syntactic and semantic regularities in language [7]. In their study Mikolov et al. created a set of analogy questions in the form "*a is to b as c is to ...*". To answer the analogy question, $y = x_b - x_a + x_c$ is computed. Where x_a , x_b , and x_c are the embedding vectors for the words a , b , and c respectively. y is the continuous space representation of the word expected as the answer. Because y is not likely to represent an existing word, the answer is considered to be the word with the embedding that has the greatest cosine similarity. The results showed better performance than prior methodologies on the SemEval-2012 Task 2: Measuring Relation Similarity. The more recent transformer-based language models have been applied to solve abstract analogy problems [14]. After pointing out the lack of attention to recognizing analogies in NLP, the study emphasized the strength and limitation of various models on psychometric analogy data sets from educational problems. The transformer models used in the study included BERT, GPT-2, RoBERTa, GPT-3. Their performances were compared against word embedding models, perplexity-based point-wise mutual information, and random answers. GPT-2 and RoBERTa were the top performer, while BERT performed worst than word embedding.

The clustering of word vectors has been used to analyze how well an embedding model's word vectors can be grouped by topic or other taxonomies. It has been applied to various domain-specific studies including business [3] and medicine [13]. Zhang et al. [17] demonstrated that incorporating word embedding

with a kernel-based k-mean clustering method provides superior performances within a topic extraction pipeline. In their study, word2vec was used to extract features from bibliometric data set, claiming the advantage to skip human feature engineering. A specifically designed k-mean clustering model is used for topic extraction. The clustering methodology is a polynomial kernel function integrated into a cosine similarity-based k-mean clustering. The performance of the proposed model was compared to different models used for topic extraction including a standard k-mean algorithm, principal component analysis, and a fuzzy c-mean algorithm. Closer to our work, an exploration of the word-class distribution in word vector spaces [11] investigated how well distribution models including Centroid-based, Gaussian Model, Gaussian Mixture Model, k-Nearest Neighbor, Support Vector Machine, and OffSet, can estimate the likelihood of a word to belong to a class. The study experimented with pre-trained vectors from Glove, and classes from the WordNet taxonomy.

While analogy tasks try to find the best word to complete a pair to be similar to another, our problem focuses on probing for a vector representation of word-pair relationships, so that they are efficiently grouped by clustering models. This by extension call for investigating the performances of clustering models for this task.

3 Methodology

3.1 Relation Vectors

To obtain a *relationship vector* from vector representations of words, we experiment with different *pooling* strategies. Here we define by pooling the mechanism by which we reduce a set of vectors representing different words, into a single one. The obtained vector will represent the relationship between the set of pooled vectors. Our first pooling strategy is to use the subtraction operator. . This strategy is derived from the linguistic regularity observation such as *king - man + woman = queen*. We can deduce *king - man = queen - woman*, where we consider the subtraction, the operator that gives a representation of the type of relationship between word vectors on both sides of the equation. Thus if v_r is the relation vector representing the relation between the word vectors v_s and v_o , we have:

$$v_r = v_s - v_o \quad (1)$$

The second strategy is to apply the absolute value function to each component of the vector resulting from the subtraction.

$$v_r = \langle |v_{o_1} - v_{s_1}|, |v_{o_2} - v_{s_2}|, \dots, |v_{o_n} - v_{s_n}| \rangle \quad (2)$$

Where v_{o_i} and v_{s_i} are respectively the i^{th} dimensional coordinate of v_o and v_s . The third consist of adding the two vectors involved in the relationship.

$$v_r = v_s + v_o \quad (3)$$

The fourth pooling strategy constitutes a vector with the coordinate obtained by taking the minimum of each dimensional coordinate of the word vectors involved in the relationship.

$$v_r = \langle \min(v_{o_1}, v_{s_1}), \min(v_{o_2}, v_{s_2}), \dots, \min(v_{o_n}, v_{s_n}) \rangle \quad (4)$$

Conversely, the fifth pooling strategy, named max pooling, consists of creating the relationship vector using the maximum of each dimensional coordinate.

$$v_r = \langle \max(v_{o_1}, v_{s_1}), \max(v_{o_2}, v_{s_2}), \dots, \max(v_{o_n}, v_{s_n}) \rangle \quad (5)$$

The last pooling strategy we use in our exploration is the average pooling.

$$v_r = \langle (v_{o_1} + v_{s_1})/2, (v_{o_2} + v_{s_2})/2, \dots, (v_{o_n} + v_{s_n})/2 \rangle \quad (6)$$

Table 1 gives the summary of the pooling strategies considered in this study.

Table 1. Pooling mechanisms for relationship vector representation.

Name	Formula
Substraction	$v_r = v_1 - v_2$
Substraction absolute value	$ v_1 - v_2 $
Addition	$v_r = v_1 + v_2$
Minimum	$v_r = \langle \min(v_{o_1}, v_{s_1}), \min(v_{o_2}, v_{s_2}), \dots, \min(v_{o_n}, v_{s_n}) \rangle$
Maximum	$v_r = \langle \max(v_{o_1}, v_{s_1}), \max(v_{o_2}, v_{s_2}), \dots, \max(v_{o_n}, v_{s_n}) \rangle$
Mean	$v_r = \langle (v_{o_1} + v_{s_1})/2, (v_{o_2} + v_{s_2})/2, \dots, (v_{o_n} + v_{s_n})/2 \rangle$

3.2 Clustering

Clustering is an active research field, that consist of grouping similar objects into sets. It can be achieved with various algorithms that differ in mechanisms employed to constitute a cluster. Although more than seventy clustering models that can be classified into nearly twenty categories are commonly used [16], we experiment with four types of the most widely used clustering mechanisms, including centroid-based, hierarchical-based, distribution-based and density-based.

Centroid-based clustering represents their cluster based on a central vector that is usually randomly selected and then optimized. k-mean [6] and its variants including k-mean++ [2] and k-medians are part of this clustering category.

Hierarchical clustering aims to build a hierarchy of clusters, either bottom-up (agglomerative) or top-down (divisive). The agglomerative approach starts by considering each data point as a cluster. Pairs of clusters are then gradually merged, moving up the hierarchy. Conversely, the divisive approach, start by considering all data points as one cluster. Clusters are then progressively split moving down the hierarchy. A measure of dissimilarity is used to determine which clusters should be merged or split. The dissimilarity is measured with a distance metric and a linkage criterion.

Distribution-based clustering assumes the data points follow a statistical distribution. While distribution based clustering models can capture statistical relationships within attributes of the data point, they work efficiently when the data distribution is known and the model parameters are set accordingly.

Density-based clustering forms clusters by grouping data points from dense areas into clusters. Such clustering algorithm allows arbitrary-shaped but they might not be precise with data sets of varying densities. Furthermore, outlier data points are considered noise. Common density-based clustering includes Density-Based Spatial Clustering of Application with Noise (DBSCAN) [5], Ordering points to identify the clustering structure (OPTICS), and Mean-shift.

4 Experiment

This section describes the data set used and selected algorithms used for experimentation. The experiments are conducted with the clustering model implementation of the Scikit-learn [9] and Scipy [15] python packages.

We use the adjusted random score to evaluate the clustering accuracy of each setting.

4.1 Datasets

We use GloVe pre-trained word vectors ¹ trained on a corpus composed of the 2014’s Wikipedia dump and the 5th edition of the English Gigaword. The training corpus has six billion tokens, four hundred thousand unique words, and is lower-cased. Our experiments use the 100 dimensions pre-trained vectors version.

The word pairs are drawn from the word pairs analogy data set of Mikolov et al. [8]. It contains 14 categories of word pairs for analogies tasks. Table 2 provides a sample of the data set word pairs for two categories.

For each word pair, their relation vector is obtained following the different pooling strategies. Figure 1 gives a 2D projection overview of the relationship vectors for each pooling strategy.

¹ <https://nlp.stanford.edu/projects/glove>

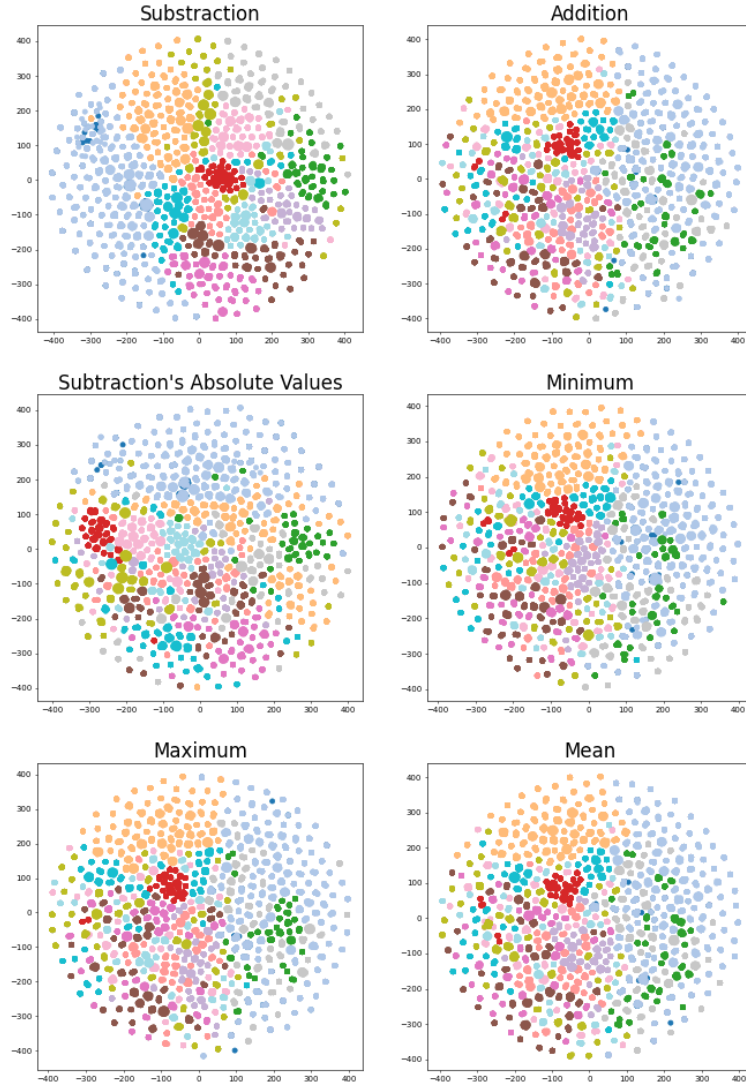


Fig. 1. TSNE 2D projections of relation vectors for different pooling strategy

Table 2. Sample of the word pairs dataset.

Countries' capital	Currencies
baghdad - iraq	japan - yen
bangkok - thailand	korea - won
beijing - china	latvia - lats
berlin - germany	lithuania - litas
bern - switzerland	macedonia - denar

4.2 K-mean

K-mean is not only the most used clustering mechanism but probably the most well-known clustering algorithm overall, because of its simple approach. The basic running steps of k-mean are the following:

1. Define the number of clusters and randomly initialize their center points.
2. Each data point is added to the cluster of its closest center point.
3. Center points are recomputed, using the *mean* of all vectors in a given cluster.
4. The two previous steps are repeated, until the center points respectively converge.

In addition to being simple, k-mean has a linear run time complexity $O(n)$. Some drawbacks are the need to specify the number of clusters, and the randomized initialization can provide different clustering results on different algorithms run. Table 3 provides the adjusted random scores for K-mean clustering.

Table 3. Adjusted random scores for K-mean clustering on the pooling strategies.

	X_{subs}	X_{add}	X_{abs}	X_{min}	X_{max}	X_{mean}
kmean	0.792549	0.363478	0.296212	0.334140	0.313051	0.363478

4.3 Gaussian mixture

The Gaussian Mixture Model is a widely used clustering model of this type. It assumes that the point in the data set follows a gaussian distribution. The model can use two parameters such as the mean and the standard deviation to describe the shape of the clusters. The parameters are found using the Expectation-Maximization optimization algorithm. The Gaussian Mixture Model clustering works as follows:

1. Randomly initialize the Gaussian distribution parameters for each cluster for the selected number of clusters.
2. Compute the probability of each data point to belong to a cluster, given the Gaussian distribution.
3. Update the set of parameters for the Gaussian distributions, based on the data point probabilities. The new parameters are computed, maximizing the probability of data points within the cluster.
4. The last two steps are iteratively repeated until the clusters' centers respectively converge.

The Gaussian mixture can be seen as a more flexible k-mean. Instead of assuming clusters with a circular shape such as k-mean, Gaussian mixture can shape ellipse-like clusters.

Table 4 provides the adjusted random score for the Gaussian mixture model clustering.

Table 4. The adjusted random score of the Gaussian Mixture Model for different pooling strategies.

	X_{subs}	X_{add}	X_{abs}	X_{min}	X_{max}	X_{mean}
gmm	0.854000	0.293000	0.351000	0.330000	0.228000	0.293000

4.4 Agglomerative Clustering

The hierarchical agglomerative clustering model can be described as follows:

1. Every point in the data set is initially considered a cluster.
2. Using a distance metric, clusters are jointly merged by pairs of the closest to one another.
3. The previous step is repeated until a unique cluster is formed. The clustering structure is then defined by choosing when to stop the clusters combination.

The dissimilarity is measured with a distance metric and a linkage criterion. We test various configurations of distance metrics as (dis)similarity measures, and linkage criterion. The distance metrics used include euclidean distance, cosine similarity, manhattan, l1, and l2. The linkage criterion is the strategy used to merge clusters at different time steps. We experiment with the following linkage criterion:

- ward: minimize the variance of the clusters being merged,
- average: merge two clusters with the minimal average distances of each observation,

- complete/maximum: merges two clusters with the smallest maximum distances between all observations of the two sets,
- single: merges two clusters with the smallest minimum distances between all observations of the two sets.

The adjusted random scores of different configurations for the linkage and distance metric parameters are available in table 5.

Table 5. The adjusted random score for different configurations of distance and linkage parameters of agglomerative clustering, for different pooling strategies

	X_{subs}	X_{add}	X_{abs}	X_{min}	X_{max}	X_{mean}
(ward, euclidean)	0.682373	0.302749	0.342485	0.317647	0.283547	0.302749
(single, euclidean)	0.006886	0.007961	0.006723	0.004333	0.006045	0.007961
(complete, euclidean)	0.502632	0.296154	0.081222	0.167146	0.299644	0.296154
(average, euclidean)	0.022881	0.293066	0.016404	0.129966	0.249915	0.293066
(single, cosine)	0.004966	0.009726	0.003661	0.008308	0.008689	0.009726
(complete, cosine)	0.695384	0.309572	0.420896	0.281730	0.356601	0.309572
(average, cosine)	0.612819	0.292957	0.273538	0.319111	0.448297	0.292957
(single, manhattan)	0.006766	0.007777	0.007989	0.005402	0.003136	0.007777
(complete, manhattan)	0.550522	0.301899	0.030703	0.273196	0.309888	0.301899
(average, manhattan)	0.021224	0.291166	0.016537	0.132257	0.269386	0.291166
(single, l1)	0.006766	0.007777	0.007989	0.005402	0.003136	0.007777
(complete, l1)	0.550522	0.301899	0.030703	0.273196	0.309888	0.301899
(average, l1)	0.021224	0.291166	0.016537	0.132257	0.269386	0.291166
(single, l2)	0.006886	0.007961	0.006723	0.004333	0.006045	0.007961
(complete, l2)	0.502632	0.296154	0.081222	0.167146	0.299644	0.296154
(average, l2)	0.022881	0.293066	0.016404	0.129966	0.249915	0.293066

4.5 DBSCAN

DBSCAN is the ubiquitous density-based clustering. The mechanism of DBSCAN can be summarized as follows [12].

1. Find points within ϵ distance of every point and identify the core points which are points with more than a minimum number of points within distance ϵ .
2. Determine the connected components of core points on the neighbor graph, excluding all non-core points.
3. Assign each non-core point to a nearby cluster within an ϵ distance neighbor, consider it a noisy point.

We experimented with different metric types including euclidean, cosine, and manhattan, as well as different values to consider points as neighbors of a core point. Table 6 provides the adjusted random score for the different experimental configurations for DBSCAN.

Table 6. The adjusted random score of DBSCAN for different pooling strategies.

	X_{subs}	X_{add}	X_{abs}	X_{min}	X_{max}	X_{mean}
(euclidean, 050)	0.028191	0.028177	0.028177	0.028177	0.028177	0.028177
(cosine, 025)	0.325427	0.217478	0.014572	0.243042	0.266950	0.217478
(cosine, 030)	0.627861	0.043688	0.000000	0.039113	0.261000	0.043688
(cosine, 050)	0.512095	0.006298	0.000000	0.003168	0.001254	0.006298
(manhattan, 050)	0.028191	0.028177	0.028177	0.028177	0.028177	0.028177

5 Discussion

5.1 Results

Our results suggest that the subtraction pooling strategy might be the best operation for word embedding-based word relationship representation, compared to the five others, experimented with, in the investigation. This finding supports the assumption of using the subtraction operator in word vector-based analogy tasks [8]. Table 7 gives the model configuration with the highest average score on each pooling strategy for different clustering models.

Table 7. Configurations with the highest score average from the different clustering models experimented with.

	X_{subs}	X_{add}	X_{abs}	X_{min}	X_{max}	X_{mean}
kmean	0.792549	0.363478	0.296212	0.334140	0.313051	0.363478
gmm	0.853806	0.293151	0.351499	0.329660	0.228340	0.293151
agglomerative: complete, cosine	0.695384	0.309572	0.420896	0.281730	0.356601	0.309572
dbscan: cosine, 025	0.325427	0.217478	0.014572	0.243042	0.266950	0.217478

In addition, the k-mean centroid-based clustering algorithm yields the highest scores for 3 of 6 pooling strategies. The best score comes from clustering relation vectors from the subtraction pooling strategy using the Gaussian Mixture

clustering model. Although the Gaussian Mixture Model is a distribution-based clustering mechanism it is built on top of the centroid-based k-means. This reinforces the superiority of centroid-based clustering in our experimental setup, followed by the agglomerative hierarchical-based clustering configured with complete linkage and cosine similarity.

5.2 Application

The ability to group representation of similar relationships between pairs of words especially named entities. Point toward an unsupervised approach to categorize relationships among words including named entities. This can in turn be used as link categorization when building knowledge graphs. Thus alleviating in some cases the need for manual data labeling for the type of links between nodes. It can go as far as providing data for graph learning models such as graph convolutional neural network that aims at including links type in addition to nodes type in their learning process.

5.3 Future Work

This work gives two pointers for further steps. One is extending the experimentation, with more word relationship representation strategy and clustering models. Additional relationship representation can include learned one by using an autoencoder on the pair of word vectors. Another direction is to derive a formal explanation of the current findings, of this exploratory study.

6 Conclusion

This study explores possibilities for a word embedding based word to word relationship representation and their clustering ability in regards to grouping similar relationships. Different relationship representations are obtained by applying basic operations on the coordinate of pairs of vectors. The subtraction pooling strategy and centroid-based clustering models tend to give better results in our exploratory setup. Further work might extend the exploration or provide a formal explanation of the findings.

References

1. Bojanowski, P., Grave, E., Joulin, A., Mikolov, T.: Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics* **5**, 135–146 (2017). https://doi.org/10.1162/tac1_a_00051
2. Bradley, P.S., Mangasarian, O.L., Street, W.N.: Clustering via concave minimization. *Advances in Neural Information Processing Systems* pp. 368–374 (1997)
3. Gohourou, D., Kurita, D., Kuwabara, K., Huang, H.H.: International business matching using word embedding. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. vol. 10191 LNAI (2017). https://doi.org/10.1007/978-3-319-54472-4_18

4. Le, Q., Mikolov, T.: Distributed representations of sentences and documents. 31st International Conference on Machine Learning, ICML 2014 **4**, 2931–2939 (2014)
5. Ling, R.F.U.o.C.: On the theory and construction of k-cluster. The Computer Journal. **15**(4), 326–332 (1972). <https://doi.org/https://doi.org/10.1093/comjnl/15.4.326>, <https://doi.org/10.1093/comjnl/15.4.326>
6. Lloyd, S.P.: Least Squares Quantization in PCM. IEEE Transactions on Information Theory **28**(2), 129–137 (1982). <https://doi.org/10.1109/TIT.1982.1056489>
7. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. In: 1st International Conference on Learning Representations, ICLR 2013 - Workshop Track Proceedings. pp. 1–12 (2013)
8. Mikolov, T., Yih, W.t., Zweig, G.: Linguistic Regularities in Continuous Space Word Representations. In: Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. pp. 746–751. Association for Computational Linguistics, Atlanta, Georgia (2013). <https://doi.org/10.3109/10826089109058901>, <https://aclanthology.org/N13-1090>
9. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
10. Pennington, J., Socher, R., Manning, C.: GloVe: Global Vectors for Word Representation. Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pp. 1532–1543 (2014). <https://doi.org/10.1080/02688697.2017.1354122>
11. Sasano, R., Korhonen, A.: Investigating Word-Class Distributions in Word Vector Spaces. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. pp. 3657–3666. Association for Computational Linguistics (2020). <https://doi.org/10.18653/v1/2020.acl-main.337>
12. Schubert, E., Sander, J., Ester, M., Kriegel, H.P., Xu, X.: DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN. ACM Transactions on Database Systems **42**(3) (2017). <https://doi.org/10.1145/3068335>
13. Suárez-Paniagua, V., Segura-Bedmar, I., Martínez, P.: Word Embedding Clustering for Disease Named Entity Recognition. Proc. of the Fifth BioCreative Challenge Evaluation Workshop pp. 299–304 (2015)
14. Ushio, A., Espinosa-Anke, L., Schockaert, S., Camacho-Collados, J.: BERT is to NLP what AlexNet is to CV: Can pre-trained language models identify analogies? ACL-IJCNLP 2021 - 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, Proceedings of the Conference pp. 3609–3624 (2021). <https://doi.org/10.18653/v1/2021.acl-long.280>
15. Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S.J., Brett, M., Wilson, J., Millman, K.J., Mayorov, N., Nelson, A.R.J., Jones, E., Kern, R., Larson, E., Carey, C.J., Polat, İ., Feng, Y., Moore, E.W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E.A., Harris, C.R., Archibald, A.M., Ribeiro, A.H., Pedregosa, F., van Mulbregt, P., SciPy 1.0 Contributors: SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods **17**, 261–272 (2020). <https://doi.org/10.1038/s41592-019-0686-2>

16. Xu, D., Tian, Y.: A Comprehensive Survey of Clustering Algorithms. *Annals of Data Science* **2**(2), 165–193 (2015). <https://doi.org/10.1007/s40745-015-0040-1>
17. Zhang, Y., Lu, J., Liu, F., Liu, Q., Porter, A., Chen, H., Zhang, G.: Does deep learning help topic extraction? A kernel k-means clustering method with word embedding. *Journal of Informetrics* **12**(4), 1099–1117 (2018). <https://doi.org/10.1016/j.joi.2018.09.004>, <https://doi.org/10.1016/j.joi.2018.09.004>